# Планирование заданий

1. Введение в планирование. Категории алгоритмов планирования. Задачи алгоритмов планирования.
2. Планирование в системах пакетной обработки данных, в интерактивных системах, в системах реального времени.
3. Управление ходом планирования. Языки управления заданиями в ОС семейства Windows

Основная цель планирования вычислительного процесса заключается в распределении времени процессора (нескольких процессоров) между выполняющимися заданиями пользователей таким образом, чтобы удовлетворять требованиям, предъявляемым пользователями к вычислительной системе. Такими требованиями могут быть, как это уже отмечалось, пропускная способность, время отклика, загрузка процессора и др.

Все виды планирования, используемые в современных ОС, в зависимости от временного масштаба, делятся на долгосрочное, среднесрочное, краткосрочное и планирование ввода-вывода. Рассматривая частоту работы планировщика, можно сказать, что долгосрочное планирование выполняется сравнительно редко, среднесрочное несколько чаще. Краткосрочный планировщик, называемый часто диспетчер (dispatcher), обычно работает, определяя, какой процесс или поток будет выполняться следующим. Ниже приведен перечень функций, выполняемых планировщиком каждого вида.

| **Вид планирования** | **Выполняемые функции** |
|---|---|
| Долгосрочное | Решение о добавлении задания (процесса) в пул выполняемых в системе |
| Среднесрочное | Решение о добавлении процесса к числу процессов, полностью или частично размещенных в основной памяти |
| Краткосрочное | Решение о том, какой из доступных процессов (потоков) будет выполняться процессором |
| Планирование ввода-вывода | Решение о том, какой из запросов процессов (потоков) на операцию ввода-вывода будет выполняться свободным устройством ввода-вывода |

Место планирования в графе состояний и переходов. В большинстве операционных систем универсального назначения планирование осуществляется *динамически* (on-line), т.е. решения принимаются во время работы системы на основе анализа текущей ситуации, не используя никаких предложений о мультипрограммной смеси. Найденное оперативно решение в таких условиях редко бывает оптимальным.

Место планирования в графе процессов

Другой тип планирования – *статический* (предварительный), может быть использован только в специализированных системах с заданным набором задач (заранее определенным), например, в управляющих вычислительных системах или системах реального времени. В этом случае статический планировщик (или предварительный планировщик) принимает решение не во время работы системы, а заранее (off-line). Результатом его работы является расписание – таблица, в которой указано, какому процессу, когда и на какое время должен быть предоставлен процессор. При этом накладные расходы ОС на исполнение расписания значительно меньше, чем при динамическом планировании.

Краткосрочный планировщик (диспетчер) реализует найденное решение, т.е. переключает процессор с одного процесса (потока) на другой. Он вызывается при наступлении события, которое может приостановить текущий процессор или предоставить возможность прекратить выполнение данного процесса (потока) в пользу другого. Примерами этих событий могут быть:

- прерывание таймера;
- прерывание ввода-вывода;
- вызовы операционной системы;
- сигналы.

Среднесрочное планирование является частью системы свопинга. Обычно решение о загрузке процесса в память принимается в зависимости от степени многозадачности (например, OS MFT, OS MVT). Кроме того, в системе с отсутствием виртуальной памяти среднесрочное планирование тесно связано с вопросами управления памятью.

Диспетчеризация сводится к следующему:

- сохранение контекста текущего потока, который требуется сменить;
- загрузка контекста нового потока, выбранного в результате планирования;
- запуск нового потока на выполнение.

Поскольку переключение контекстов существенно влияет на производительность вычислительной системы, программные модули ОС выполняют эту операцию при поддержке аппаратных средств процессора.

В мультипрограммной системе поток (процесс, если операционная система работает только с процессами) может находиться в одном из трех основных состояний:

- **выполнение** – активное состояние потока, во время которого поток обладает всеми необходимыми ресурсами и непосредственно выполняется процессором;
- **ожидание** – пассивное состояние потока, находясь в котором, поток заблокирован по своим внутренним причинам (ждет осуществления некоторого события, например, завершения операции ввода-вывода, получения сообщения от другого потока или освобождения какого-либо необходимого ему ресурса);
- **готовность** – также пассивное состояние потока, но в этом случае поток заблокирован в связи с внешним по отношению к нему обстоятельством (имеет все требуемые ресурсы, готов выполняться, но процессор занят выполнением другого потока).

В течение своей жизни каждый поток переходит из одного состояния в другое в соответствии с алгоритмом планирования потоков, принятом в данной операционной системе.

В состоянии выполнения в однопроцессорной системе может находиться не более одного потока, а в остальных состояниях – несколько. Эти потоки образуют очереди, соответственно, ожидающих и готовых потоков. Очереди организуются путем объединения в списки описателей отдельных потоков. С самых общих позиций все множество алгоритмов планирования можно разделить на два класса: вытесняющие и не *вытесняющие* алгоритмы планирования.

Не вытесняющие (non-preemptive) алгоритмы основаны на том, что активному потоку позволяется выполняться, пока он сам, по своей инициативе, не отдает управление операционной системе, для того чтобы она выбрала из очереди готовый к выполнению поток.

Вытесняющие (preemptive) алгоритмы – это такие способы планирования потоков, в которых решение о переключении процессора с выполнения одного потока на выполнение другого потока принимается операционной системой, а не активной задачей.

В первом случае механизм планирования распределяется между операционной системой и прикладными программами. Во втором случае функции планирования потоков целиком сосредоточены в операционной системе.

Недостатком первого типа алгоритмов планирования является необходимость разработки такого приложения, которое будет "дружественным" по отношению к другим

выполняемым одновременно с ним программам. Для этого в приложении должны быть предусмотрены частные передачи управления операционной системе. Крайним проявлением недружественности приложения является его зависание, которое приводит к общему краху системы. Потому распределение функций планирования между ОС и приложениями достаточно сложно в программистском отношении и используется, как правило, в специализированных системах с фиксированным набором задач. В то же время существенным преимуществом невытесняющего планирования является более высокая скорость переключения потоков.

Однако почти во всех ОС (UNIX, Windows NT/2000/2003, OS/2, VAX/VMS и др.) реализованы вытесняющие алгоритмы планирования. В основе многих таких алгоритмов лежит концепция квантования. В соответствии с ней каждому потоку поочередно для выполнения предоставляется ограниченный непрерывный период процессорного времени – квант.

Смена активного потока происходит, если:

- поток завершается и покинул систему;
- произошла ошибка;
- поток перешел в состояние ожидания;
- исчерпан квант времени, отведенный данному потоку.

Поток, который исчерпал свой квант, переводится в состояние готовности и ожидает, когда ему будет предоставлен новый квант процессорного времени, а на выполнение в соответствии с определенным правилом выбирается новый поток из очереди готовых потоков.

Кванты, выделяемые потокам, могут быть равными или различными (типичное значение десятки – сотни мс). Например, первоначально каждому потоку назначается достаточно большой квант, а величина каждого следующего кванта уменьшается до некоторой заранее заданной величины. В таком случае преимущество получают короткие задачи, которые успевают выполняться в течение первого кванта (второго и т.д.), а длительные вычисления будут проводиться в фоновом режиме.

Некоторые потоки, получив квант времени, используют его не полностью, например, из-за необходимости выполнить ввод или вывод данных. В результате может возникнуть ситуация, когда потоки с интенсивным вводом-выводом используют только небольшую часть выделенного им процессорного времени. Можно исправить эту "несправедливость", изменив алгоритм планирования, например, так: создать две очереди потоков, очередь 1 – для потоков, которые пришли в состояние готовности в результате исчерпания кванта времени, и очередь 2 – для потоков, у которых завершилась операция ввода-вывода. При выборе потока для выполнения сначала просматривается вторая очередь, и если она пуста, квант выделяется потоку из первой очереди.

Отметим три замечания об алгоритмах, основанных на квантовании.

Первое. Переключение контактов потоков связано с потерями процессорного времени, которые не зависят от величины кванта, но зависят от частоты переключения. Поэтому чем больше квант, тем меньше суммарные затраты процессорного времени на переключение потоков.

Второе. С увеличением кванта может быть ухудшено качество обслуживания пользователей, связанное с ростом времени реакции системы.

Третье. В алгоритмах, основанных на квантовании, ОС не имеет никаких сведений о решаемых задачах (длинные или короткие, интенсивен "ввод-вывод" или нет, важно быстрое исполнение или нет и т.п.). Дифференциация обслуживания при квантовании базируется на "истории существования" потока в системе.

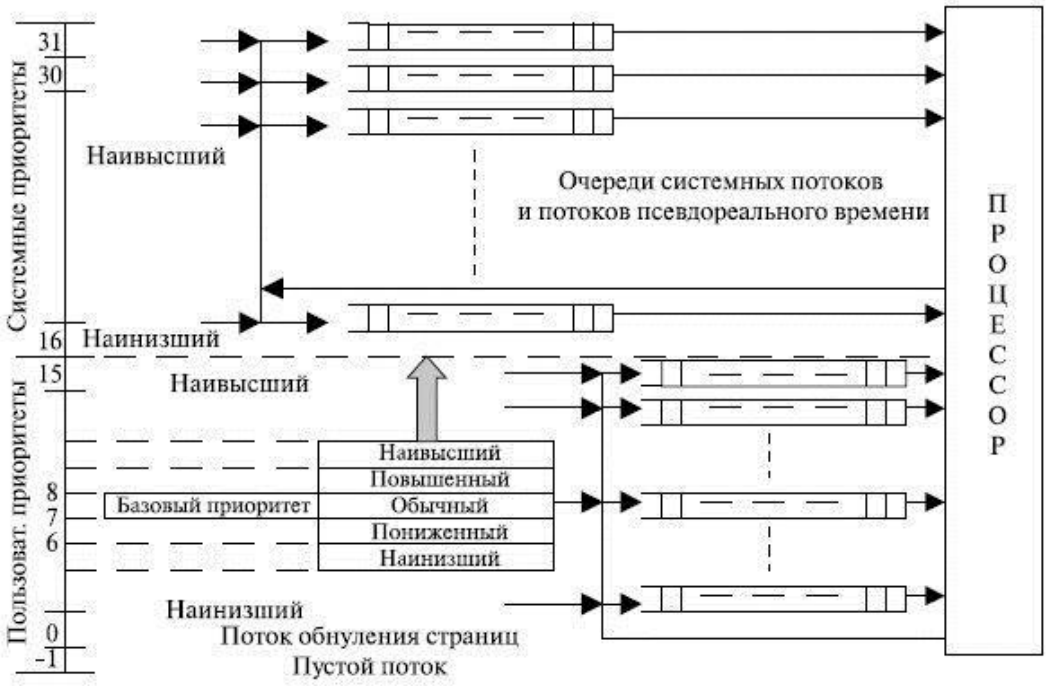
Важной концепцией, лежащей в основе многих вытесняющих алгоритмов планирования, является приоритетное обслуживание. Оно предполагает наличие у потоков некоторой изначально известной характеристики – приоритета, на основании которого определяется порядок выполнения потоков. Чем выше приоритет, тем выше привилегии потока, тем меньше времени поток находится в очередях. Приоритет задается числом (целым или дробным, положительным или отрицательным).

В большинстве ОС, поддерживающих потоки, приоритет потока связан с приоритетом процесса, в рамках которого выполняется поток. Приоритет процесса назначается операционной системой при его создании, его значение включается в описатель процесса и используется при назначении приоритета потоком этого процесса. При назначении приоритетов вновь созданному процессу ОС учитывается, является ли этот процесс системным или прикладным, каков статус пользователя, запустившего процесс (администратор, пользователь, часть и т.п.), было ли явное указание пользователя на присвоение процессу определенного уровня приоритета. Поток может быть инициирован не только по команде пользователя, но и в результате выполнения системного вызова другим потоком. В этом случае ОС учитывает значения параметров системного вызова.

Изменения приоритета могут происходить по инициативе самого потока, когда он обращается с соответствующим вызовом к ОС, или по инициативе пользователя, когда он выполняет соответствующую команду. Кроме этого, сама ОС может изменить приоритеты потоков в зависимости от ситуации, складывающейся в системе. В последнем случае приоритеты называются динамическими в отличие от неизменяемых, фиксированных приоритетов. Возможности пользователей влиять на приоритеты процессов и потоков ограничены ОС. Обычно это разрешается администраторам, и то в определенных пределах. В большинстве случаев ОС присваивает приоритеты потокам по умолчанию.

Существует две разновидности приоритетного планирования: с относительными и абсолютными приоритетами. В обоих случаях на выполнение выбирается поток, имеющий наивысший приоритет. Но определение момента смены активного потока решается по-разному. В системах с относительными приоритетами активный поток выполняется до тех пор, пока он сам не покинет процессор, перейдя в состояние ожидания (по вводу-выводу, например), или не завершится, или не произойдет ошибка. В системах с абсолютными приоритетами выполнение активного потока прерывается еще и по причине появления потока, имеющего более высокий приоритет, чем у активного потока. В этом случае прерванный поток переходит в состояние готовности.

В качестве примера рассмотрим организацию приоритетного обслуживания в Windows 2000/2003/XP/Vista. Здесь приоритеты организованны в виде двух групп, или классов: реального времени и переменные. Каждая из групп состоит из 16 уровней приоритетов (рис. 5.10). Потоки, требующие немедленного внимания, находятся в классе реального времени, который включает такие функции, как осуществление коммуникаций и задачи реального времени [17].




Планирование в Windows

В целом, поскольку W2K использует вытесняющий планировщик с учетом приоритетов, потоки с приоритетами реального времени имеют преимущество по отношению к прочим потокам. В однопроцессорной системе, когда становится готовым к выполнению поток с более высоким приоритетом, чем выполняющийся в настоящий момент, текущий поток вытесняется и начинает выполняться поток с более высоким приоритетом.

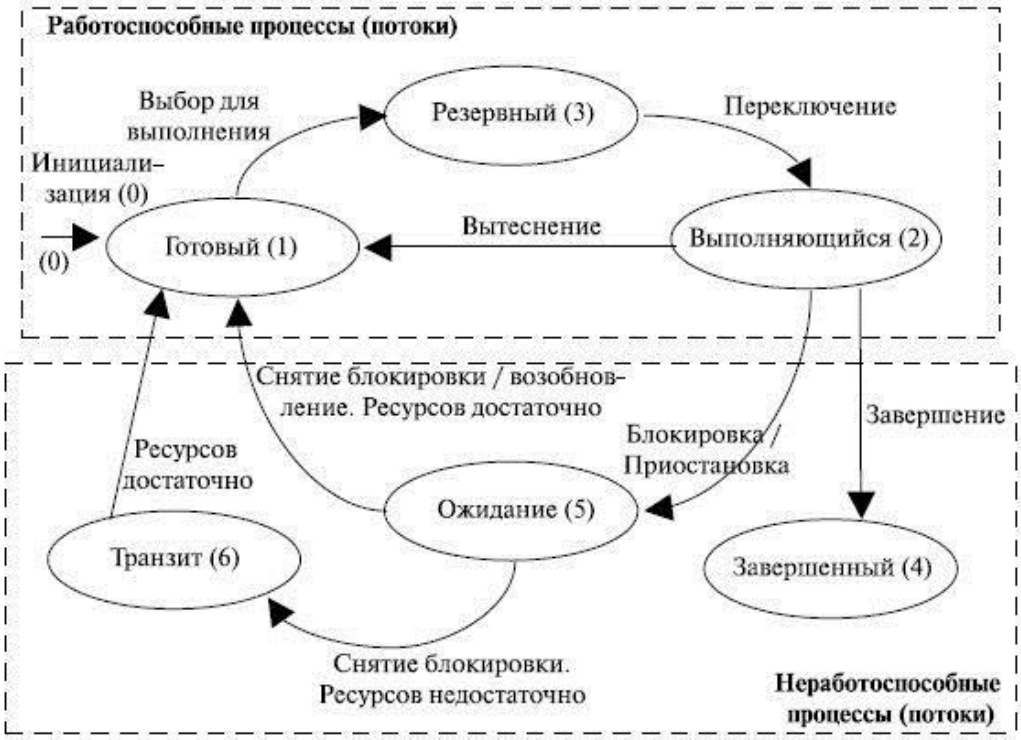
Приоритеты из разных классов обрабатываются несколько по-разному. В классе приоритетов реального времени все потоки имеют фиксированный приоритет (от 16 до 31), который никогда не изменяется, и все активные потоки с определенным уровнем приоритета располагаются в круговой очереди данного класса (tкв=20 мс для W2K Professional, 120 мс – для однопроцессорных серверов).

В классе переменных приоритетов поток начинает работу с базового приоритета процесса, который может принимать значение от 1 до 15. Каждый поток, связанный с процессом имеет, свой базовый приоритет, равный базовому приоритету процесса, или отличающийся от него не более чем на 2 уровня в большую или меньшую сторону. После активации потока его динамический приоритет может колебаться в определенных пределах – он не может упасть ниже наименьшего базового приоритета данного класса, т.е. 15 (для потоков с приоритетом 16 и выше никогда не делается никаких изменений приоритетов).

Когда же увеличивается приоритет потока? Во-первых, когда завершается операция ввода-вывода и освобождается ожидающий ее поток, его приоритет увеличивается, чтобы дать шанс этому потоку запуститься быстрее и снова запустить операцию ввода-вывода. Суть в том, чтобы поддержать занятость устройств ввода-вывода. Величина, на которую увеличивается приоритет, зависит от устройства ввода-вывода. Как правило, это 1 – для диска, 2 – для последовательной линии, 6 – для клавиатуры и 8 – для звуковой карты.

Во-вторых, если поток ждал семафора, мьютекса или другого события, то когда он отпускается, к его приоритету добавляется 2 единицы, если это поток переднего плана (т.е. управляет окном, к которому направляется ввод с клавиатуры), и одна единица – в противном случае. Таким образом, интерактивный процесс получает преимущество перед большим количеством других процессов. Наконец, если поток графического интерфейса пользователя просыпается, потому что стал доступен оконный ввод, он также получает прибавку к приоритету.

Эти увеличения приоритета не вечны. Они незамедлительно вступают в силу, но если поток использует полностью свой следующий квант, он теряет один пункт приоритета. Если он использует еще квант, то перемещается еще на уровень ниже и т.д. вплоть до своего базового уровня.

Последняя модификация алгоритма планирования W2K заключается в том, что когда окно становится окном переднего плана, все его потоки получают более длительные кванты времени (величина прибавки хранится в системном реестре).

Поток, созданный в системе, может находиться в одном из 6 состояний в соответствии с графом, приведенным на рисунке